API-Schlüssel für den KI-Agenten

Lernunterlage: Maschinen anmelden, mehrere Anmeldeverfahren, Lesen schützen

1 Das Problem: Ein Programm ist kein Mensch

Das Login aus der letzten Lernunterlage ist für Menschen gebaut: E-Mail und Passwort eintippen, im Browser ein Cookie bekommen, alle 15 Minuten im Hintergrund ein neues Access Token holen. Dein KI-Agent in n8n läuft aber ohne Browser, ohne Cookie-Speicher und ohne dass jemand ein Passwort eintippt. Er braucht etwas Einfacheres.

	Mensch im Browser (Frontend)	Programm (n8n-Agent)
Wer meldet sich an?	Ein Benutzer mit E-Mail und Passwort	Ein Dienst, der selbstständig Anfragen schickt
Was schickt er mit?	Access Token (JWT), 15 Minuten gültig	Einen festen geheimen Schlüssel
Erneuern?	Refresh-Token im Cookie, Rotation	Nicht nötig, der Schlüssel bleibt gleich
Wo steckt es?	Local Storage und HTTP-only Cookie	In n8n (Zugangsdaten) und in den User Secrets des Backends
Was darf er?	Je nach Rolle: User liest, Admin schreibt	Nur lesen

Was ist ein API-Schlüssel?

Ein API-Schlüssel (API key) ist ein langer, zufälliger Text, den nur der Server und der berechtigte Client kennen. Er funktioniert wie ein Passwort für ein Programm: Wer ihn schickt, wird hereingelassen. Der Server prüft nur, ob er stimmt. Er weiß nicht, *wer* ihn schickt.

> **Merksatz:** Ein API-Schlüssel ist ein Passwort für Maschinen. Er muss genauso geheim bleiben wie ein Passwort, und er verfällt nicht von selbst.

2 So macht es Home Control

Der Agent hängt an jede Anfrage einen zusätzlichen Header an:

```
GET /api/devices HTTP/1.1
X-Api-Key: <der geheime Schlüssel>
```

Das Backend vergleicht den Header mit dem Schlüssel aus seinen Einstellungen. Stimmt er, ist die Anfrage angemeldet (als „Agent“, Rolle ohne Schreibrechte). Stimmt er nicht, kommt 401.

Teil	Datei	Aufgabe
Schlüssel erzeugen	agent-schluessel-einrichten.sh	Zufälliger Schlüssel, Speicherung in den User Secrets, einmalige Anzeige zum Kopieren
Einstellungen	Auth/AgentOptions.cs	Liest Agent:ApiKey; Mindestlänge 32 Zeichen
Prüfen	Auth/ApiKeyAuthenticationHandler.cs	Liest den Header, vergleicht, baut daraus die Identität „Agent“
Regel „Lesen“	Auth/AuthPolicies.cs	CreateReadingPolicy: angemeldet per JWT **oder** API-Schlüssel

Teil	Datei	Aufgabe
Schalter	appsettings.json	Auth:RequireLoginForReading (Standard false)
Verdrahtung	Program.cs, Endpoints/AuthEndpoints.cs	Zweites Verfahren registrieren, Fallback-Regel setzen, login & Co. offen lassen

Wo liegt der Schlüssel?

- **Backend:** .NET User Secrets (`Agent:ApiKey`), liegt außerhalb des Projektordners, nie in Git.
- **n8n:** in den Zugangsdaten (Credentials, Typ „Header Auth“), nicht im Text des Workflows. Sonst steht der Schlüssel in Exporten und Screenshots des Workflows.
- **Nicht:** in der Konfigurationsdatei im Repo, in Chats, in Commit-Nachrichten.

3 Mehrere Anmeldeverfahren in ASP.NET Core

Bisher kannte das Backend nur ein Verfahren: JWT-Bearer. Jetzt gibt es zwei. In ASP.NET Core heißt so ein Verfahren **Authentication Scheme**. Jedes Scheme hat einen Namen und einen Handler, der eine Anfrage anschaut und sagt: „Ich kenne den Absender“, „Das ist ein Fehler“ oder „Dafür bin ich nicht zuständig“.

Scheme	Handler schaut auf	Ergebnis bei …
Bearer (JWT)	Header Authorization: Bearer …	gültigem Token: angemeldet · kaputtem Token: Fehler · ohne Header: nicht zuständig
ApiKey	Header X-Api-Key	richtigem Schlüssel: angemeldet · falschem: Fehler · ohne Header: nicht zuständig

Drei mögliche Ergebnisse eines Handlers

Ergebnis	Bedeutung	Im Code des Handlers
Success	Absender erkannt, Identität (Claims) liegt vor	AuthenticateResult.Success(ticket)
Fail	Es wurde etwas geschickt, aber es ist falsch	AuthenticateResult.Fail(…)
NoResult	Nichts für dieses Verfahren da, ein anderes darf es versuchen	AuthenticateResult.NoResult()

Wichtig ist **NoResult**: Ein Frontend-Aufruf hat keinen X-Api-Key-Header. Der ApiKey-Handler sagt dann „nicht zuständig“, statt die Anfrage abzulehnen, und das JWT-Verfahren kommt zum Zug.

Welches Scheme wird wann geprüft?

Ohne weitere Angabe prüft ASP.NET nur das **Default-Scheme** (bei uns Bearer). Damit der API-Schlüssel überhaupt angeschaut wird, muss eine Regel ihn ausdrücklich nennen:

```
// Kleines fremdes Beispiel (Bibliotheksverwaltung), nicht aus Home Control:
var policy = new AuthorizationPolicyBuilder("Bearer", "ApiKey")
    .RequireAuthenticatedUser()      // egal ob per Token oder per Schlüssel
    .Build();
```

Diese Regel heißt in Home Control `CreateReadingPolicy`. Die Admin-Regel nennt dagegen kein ApiKey-Scheme. Deshalb kann der Schlüssel nie schreiben: Für die Admin-Endpunkte wird er gar nicht erst angeschaut, die Anfrage bleibt „nicht angemeldet“ (401).

Anfrage	Regel	Ergebnis
GET /api/rooms mit JWT eines Users	Lesen (Bearer + ApiKey)	200
GET /api/rooms mit X-Api-Key (richtig)	Lesen (Bearer + ApiKey)	200
GET /api/rooms mit X-Api-Key (falsch)	Lesen (Bearer + ApiKey)	401
GET /api/rooms ohne alles (Schalter an)	Lesen (Bearer + ApiKey)	401
POST /api/rooms mit X-Api-Key	AdminOnly (nur Bearer, Rolle Admin)	401: Schlüssel zählt hier nicht
POST /api/rooms mit JWT eines Users	AdminOnly	403: angemeldet, aber kein Admin
POST /api/rooms mit JWT eines Admins	AdminOnly	201

4 Vergleich in konstanter Zeit

Ein normaler Textvergleich (`a == b`) hört beim ersten Unterschied auf. Stimmt schon das erste Zeichen nicht, ist er sofort fertig. Stimmen die ersten zehn, dauert er etwas länger. Ein Angreifer könnte die Antwortzeit sehr genau messen und so Zeichen für Zeichen den Schlüssel erraten. Das nennt man **Timing-Angriff**.

Gegenmittel: ein Vergleich, der **immer gleich lange** dauert. In .NET gibt es dafür `CryptographicOperations.FixedTimeEquals`. Das braucht gleich lange Eingaben. Deshalb werden beide Seiten vorher mit SHA-256 gehasht: Jeder Hash ist 32 Byte lang, egal wie lang der Schlüssel war, und die Länge des echten Schlüssels verrät sich auch nicht.

```
// Fremdes Beispiel: zwei Texte in konstanter Zeit vergleichen
bool Gleich(string a, string b) =>
    CryptographicOperations.FixedTimeEquals(
        SHA256.HashData(Encoding.UTF8.GetBytes(a)),
        SHA256.HashData(Encoding.UTF8.GetBytes(b)));
```

> **Merksatz:** Geheimnisse (Schlüssel, Tokens, Hashes) vergleicht man nie mit ==, sondern in konstanter Zeit.

5 Lesen schützen: der Schalter

Bisher darf jeder im Netz alle GET-Endpunkte lesen (Räume, Geräte, Reports, Tuya, Blink). Das war Absicht, damit der Agent nicht kaputtgeht. Mit dem API-Schlüssel kann man das jetzt schließen. Dafür gibt es die **Fallback-Regel**.

Begriff	Bedeutung
Fallback-Regel (FallbackPolicy)	Gilt für jeden Endpunkt, der keine eigene Regel hat. Hier: „angemeldet per JWT oder API-Schlüssel“.
RequireAuthorization(...)	Eigene Regel an einem Endpunkt. Sie hat Vorrang vor dem Fallback.
AllowAnonymous()	Ausnahme: dieser Endpunkt bleibt auch mit Fallback offen.

Offen bleiben müssen **register, login, refresh und logout**. Wer sich erst anmelden will, hat noch kein Token, sonst käme niemand je hinein. Ebenso `GET /` und im Development die API-Dokumentation.

Der Schalter heißt `Auth:RequireLoginForReading` und steht standardmäßig auf **false**. Ist er aus, ändert sich nichts. Das ist der sichere Weg, so etwas einzuführen: erst den neuen Weg bauen,

dann umlegen.

Reihenfolge beim Einschalten

1. Schlüssel erzeugen: `./agent-schluessel-einrichten.sh` und den angezeigten Schlüssel kopieren.
2. Backend neu starten, damit es den Schlüssel kennt.
3. In n8n den Header `X-Api-Key` bei den Home-Control-Knoten eintragen (Credentials).
4. Testen: Bekommt der Agent noch Daten?
5. Erst dann den Schalter auf `true` setzen und neu starten.

> **Warum diese Reihenfolge?** Schaltest du zuerst um und trägst den Schlüssel danach ein, bekommt der Agent dazwischen nur 401 und du suchst den Fehler an der falschen Stelle. Immer erst das Neue anbieten, dann das Alte abschalten.

6 Grenzen des Verfahrens

Grenze	Was bedeutet das?	Möglicher nächster Schritt
Schlüssel läuft nie ab	Ein gestohlener Schlüssel gilt, bis du ihn austauschst	Regelmäßig neu erzeugen; Skript überschreibt den alten
Nur ein Schlüssel	Alle Clients teilen ihn; man sieht nicht, wer gelesen hat	Ein Schlüssel pro Client, in der Datenbank gehasht
Klartext im Netz	Über HTTP ist der Header mitlesbar	Nur über Tailscale oder HTTPS (z. B. tailscale serve) benutzen
Liest alles	Der Schlüssel öffnet alle GET-Endpunkte, auch Blink	Eigene Rollen oder Bereiche pro Schlüssel
Kein Rate Limit	Raten ist erlaubt, nur sehr unwahrscheinlich (48 Zufallsbytes)	Anfragenbegrenzung einbauen
Schlüssel im Log?	Header sollten nicht in Logs landen	Logging-Einstellungen prüfen

Warum ist Raten trotzdem praktisch aussichtslos? Das Skript erzeugt 48 Zufallsbytes aus dem kryptografischen Zufallsgenerator, das sind 384 Bit. Zum Vergleich: Ein 8-stelliges Passwort aus Buchstaben und Ziffern hat rund 48 Bit.

7 Kurzantworten für Rückfragen

Frage	Antwort in einem Satz
Warum kein Login mit Passwort für den Agenten?	Ein Dienst ohne Browser müsste Cookies und die Token-Erneuerung alle 15 Minuten selbst nachbauen; ein fester Schlüssel ist dafür viel einfacher.
Warum ist der Schlüssel weniger sicher als ein JWT?	Er verfällt nicht und gehört nicht einer Person; dafür darf er nur lesen und lässt sich jederzeit austauschen.
Warum darf der Schlüssel nicht schreiben?	Schreibende Endpunkte prüfen die Admin-Rolle im JWT; das ApiKey-Scheme wird dort gar nicht angeschaut.
Was ist ein Authentication Scheme?	Ein benanntes Anmeldeverfahren mit einem Handler, der eine Anfrage prüft.
Was macht NoResult?	Der Handler ist nicht zuständig; ein anderes Verfahren darf es versuchen.

Frage	Antwort in einem Satz
Was ist die Fallback-Regel?	Die Regel für alle Endpunkte ohne eigene Regel; AllowAnonymous nimmt einzelne aus.
Warum FixedTimeEquals?	Normale Vergleiche verraten durch ihre Laufzeit, wie viele Zeichen stimmen (Timing-Angriff).
Warum erst ein Schalter?	So bricht beim Einführen nichts; der Agent bekommt erst seinen Schlüssel, dann wird umgelegt.
Wo liegt der Schlüssel?	Backend: User Secrets. n8n: Credentials. Nie im Repo.
Was, wenn der Schlüssel geleakt ist?	Skript erneut ausführen, Backend neu starten, neuen Schlüssel in n8n eintragen; der alte ist sofort ungültig.

8 Merksätze

- Menschen melden sich mit Passwort und Token an, **Maschinen mit einem API-Schlüssel**.
- Ein API-Schlüssel ist ein Passwort: geheim halten, nicht in Git, nicht in Workflows.
- Jedes **Authentication Scheme** hat einen Handler: Success, Fail oder NoResult.
- Eine Regel prüft nur die Schemes, die sie nennt. Der Admin-Regel fehlt ApiKey, deshalb kann der Schlüssel nie schreiben.
- **Fallback-Regel** = Standard für alle Endpunkte ohne eigene Regel, **AllowAnonymous** = Ausnahme.
- Geheimnisse vergleicht man in **konstanter Zeit** (FixedTimeEquals).
- Erst das Neue anbieten, dann das Alte abschalten: **Schalter** statt Hauruck.
- 401 = nicht (korrekt) angemeldet, 403 = angemeldet, aber nicht berechtigt.